Rapport d'experience — model ORDER 59

Campagne executee le 20 septembre 2026

Contents

1 Environnement **2**

2 Campagne exacte **2**

3 Pipeline causal **2**

4 Equations et fonctions associees **3**

5 Resultats principaux **3**

6 Chess et absence d'oracle **4**

7 Ablations **4**

8 Memoire **4**

9 Capacite **4**

10 Transfert **4**

11 Anti-triche et tests **4**

12 Checkpoints **4**

13 Interruption et contrainte de calcul **4**

14 Traceabilite vers les sources **5**

15 Conclusion experimentale **5**

Abstract

Ce rapport decrit seulement les executions archivees. Il donne l'environnement, les commandes, les configurations, le pipeline, les equations utilisees dans le code, les resultats, les ablations, les controles d'integrite et les interruptions. Les resultats negatifs sont conserves.

1 Environnement

Python 3.13. La machine d'execution dispose de NumPy, PyTorch, pytest et pdflatex. Le package O59 est configure pour le layout `src/` via setuptools. Installation :

```
cd /tmp/order59_work/model_order_59
python -m pip install -e . --no-build-isolation -q
pytest -q
```

La passe finale de `pytest -q` sur la release corrective retourne `24 passed`. Une seconde passe a produit le meme resultat avant le gel. La couverture du noyau est de 87% des lignes instrumentees.

2 Campagne exacte

La commande complete est :

```
cd /tmp/order59_work/audit_order_59/scripts
python run_experiments.py
```

Les campagnes hard finales utilisent un seed 59059. La campagne multi-familles utilise 4 s, 30 pas, 20000 evaluations, 64 branches et 120 programmes candidats. La campagne chess longue utilise 60 s, 10 pas agent, 100000 evaluations, 4096 branches et 400 programmes candidats; chaque position est lancee comme shard independant. Les sorties finales sont archivees dans l audit O59 hard.

3 Pipeline causal

Le chemin execute est :

$$\begin{aligned} & O_t \to P_t \to I_t \to \Pi_t \to D_t \to M_t \\ & \quad \to \{probe, constraint, predict, imagine, synth, transfer, verify\} \\ & \quad \to A_t \to E \to O_{t+1}. \end{aligned}$$

Plus concretement :

1. `GenericPerceiver` transforme l'observation brute en `Observation`.
2. `GenericProbeFactory` construit des sondes generiques ; `build_partition` construit les fibres.
3. Les collisions servent au calcul de D_{obs}.
4. `MetaController` choisit une ressource de raisonnement.
5. `EAAttention` et `NeuralProbeController` orientent le choix de probe.
6. `ConstraintInducer` fabrique des contraintes a partir des interactions.
7. `PredictiveModel` met a jour le support et le modele lineaire.
8. `ImaginationEngine` produit une hypothese et `multi_step` la simule dans le monde interne.
9. `ProgramSynthesizer` enumere des AST et `ProgramVM` les execute sans code arbitraire.
10. Le score final des actions combine feedback empirique, prediction, nouveaute, contraintes induites, signal neural conditionne, programme execute et resultat d imagination action-specifique. Une verification independante rejette les propositions non admissibles avant action.
11. `env.act` est la seule source de transition reelle ; le resultat est ensuite memoire et monde-model.

4 Equations et fonctions associees

Empreinte :

$$I_P(x) = (p_1(x), \ldots, p_k(x)).$$

Suffisance :

$$I_P(x) = I_P(y) \Rightarrow A(x) = A(y).$$

Factorisation :

$$A = \bar{A} \circ I_P.$$

Raffinement :

$$I_{P+g}(x) = I_{P+g}(y) \Rightarrow I_P(x) = I_P(y),$$

donc $[x]_{P+g} \subseteq [x]_P$.

Seuil : la paire reste dans la meme fibre jusqu'au premier niveau qui la separe. Test du minimum-double :

$$\Theta(x, z) \geq \min\{\Theta(x, y), \Theta(y, z)\}.$$

Ultrametrique :

$$d(x, y) = e^{-\Theta(x,y)} \quad (x \neq y).$$

Prediction empirique :

$$\widehat{P}(s' \mid s, a) = \frac{N(s, a, s')}{\sum_u N(s, a, u)}.$$

Composition imaginative :

$$H \to W(H) \to \widehat{O}_H \to \Delta_H \to score(A).$$

Deficit :

$$D = (D_{obs}, D_{cap}, D_{res}, D_{search}, D_{transfer}, D_{constraint}).$$

Les fonctions correspondantes sont dans `core.py`, `agent.py`, `prediction.py`, `imagination.py`, `constraints.py`, `geometry.py` et `latent.py`.

5 Resultats principaux

Experience	Statut	Score	Pas	Instruments observables
HiddenPermutation	PARTIAL	0.4	12	21 probes, 1 program, 576 evaluations VM
Sequence	PARTIAL	0.142857	7	6 probes, 1 program, 12 evaluations VM
Maze	PARTIAL	0.0	12	21 probes, 1 program, 36 evaluations VM
VisualPattern	PARTIAL	0.0	7	6 probes, 1 program, 8 evaluations VM
Chess mate-in-one	PARTIAL	0.0	2	1 probe, imagination active
Chess open	PARTIAL	0.0	2	1 probe, imagination active

Les six scores sont des observations sous les budgets de cette campagne. Aucun n'est converti en limite theorique.

6 Chess et absence d'oracle

L'agent recoit `board`, `turn` et `last`. Il fabrique des actions a partir du plateau observe et d'une notation de coordonnees. La liste des coups legaux n'est jamais fournie au modele : l adaptateur expose seulement un alphabet d actions generique (toutes les coordonnees UCI candidates), tandis que la legalite est evaluee uniquement dans l adaptateur. Aucun score de position ou moteur chess n'est injecte dans l'agent.

7 Ablations

Le script historique execute 21 conditions : chacun des sept flags `use_probes`, `use_imagination`, `use_synthesis`, `use_memory`, `use_world_model`, `use_neural`, `use_constraints` est coupe sur Sequence, Maze et Visual. Ces lignes sont dans `main_results.json`. L'interpretation doit comparer la variation reelle ; elle ne doit pas conclure qu'une brique est inutile lorsqu'une suite ne la sollicite pas.

8 Memoire

La repetition de SequenceEnv donne 0.2 a l'episode 2 avec memoire et 0.2 au controle sans memoire. Le retrieval de programme est implemente, mais aucun gain causal n'est observe dans cette experience. Ce resultat negatif est conserve.

9 Capacite

Le gate de capacite rejette l'exemple dont le cout `on` est superieur au cout `off` (`causal_delta=-1`). La promotion n'est donc jamais declarative.

10 Transfert

Le decouvreur de transfert retrouve la correspondance structurale `c->z`, `a->x`, `b->y`, score 1.0. Une certification independante sur deux cas retourne VERIFIED. Le mecanisme est general dans sa forme de recherche par signature, mais la campagne ne constitue pas une preuve de transfert ouvert universel.

11 Anti-triche et tests

Le test `test_anticheat.py` applique `StaticAuditor`. Il interdit `eval`, `exec`, `os.system`, dispatch explicite sur benchmark/task/family et motif de tables de solutions. `ProgramVM` n'execute pas de code Python dynamique.

12 Checkpoints

Les checkpoints C0–C18 historiques sont materialises dans l'audit. Chaque checkpoint contient au minimum : identifiant, etat, hash de l'etape, fichiers ou fonctions touches, tests executés, anomalies et diff conceptuel.

13 Interruption et contrainte de calcul

Un essai intermediaire a 140 pas et horizon plus long a depasse le timeout global. L'analyse a relie cet incident au cout cumulatif du scoring de programmes et de l'entrainement neural. La

campagne finale borne ces deux couts. L'incident reste dans `logs/experiment_interruption.txt` et est traite comme contrainte de ressource mesuree.

14 Traceabilite vers les sources

Le concept d'empreinte, de fibre, de fermeture, de seuil, de minimum-double, de latent canonique, d'identite et de reflexion provient principalement du DDD O5. Les structures executables de fibres, contraintes et recherche proviennent des models O55/O56. Les extensions actives, compilateur, transfert conditionnel, anti-cheat et corrections proviennent de O58. La matrice de correspondance complete est dans `source_audit/implementation_traceability.json`.

15 Conclusion experimentale

La release O59.0.2 verifie que le chemin de construction de representation et plusieurs composants avances sont appeles pendant les episodes : sondes et reconstruction de representation sont executees pendant les episodes, les hypotheses d'imagination passent par une simulation interne, et les programmes synthetises sont executes dans une VM. Les performances restent partielles sous les budgets utilises, y compris chess. Ce constat ne permet pas de conclure a une limite de generalite ; il definit simplement l'etat mesure de la campagne.